POLYTECH INTL

Mini-projet Web Sémantique

Cinéma tunisien

Modélisation RDFS, requêtes SPARQL, inférence et web service REST

Classe : BDBI 3 - BDIA 3

Enseignante : Mme Ahlem TAYACHI

Réalisé par : Ahmed TOUJANI

Date de validation : 04/11/2026

1. Introduction

L'objectif de ce projet est d'exposer un jeu de données RDF sous la forme d'un web service. Le thème choisi est le cinéma tunisien : un réalisateur réalise un film, un acteur joue dans un film, un film a un genre et passe dans des salles. Les données sont illustratives.

Le travail suit les quatre étapes demandées :

- **Étape 1** : modélisation du vocabulaire avec RDFS ;
- **Étape 2** : requêtes SPARQL répondant à des questions métier ;
- **Étape 3** : inférence RDFS et comparaison des résultats avec et sans inférence ;
- **Étape 4** : service REST (Flask) renvoyant du JSON.

Technologies : Python 3, rdflib (graphe RDF et SPARQL), owlrl (inférence RDFS), Flask (REST). Chemin d'une information : les fichiers `.ttl` sont lus par `graph.py` pour former un graphe ; `queries.py` l'interroge avec SPARQL ; `app.py` expose les résultats en JSON.

2. Étape 1 : modélisation RDFS

Le vocabulaire est défini dans `schema.ttl` avec l'espace de noms `http://example.org/cinema#` (préfixe `cin`). Les données sont dans `data.ttl` : 4 films, 3 réalisateurs, 7 acteurs, 3 genres et 3 salles. RDF décrit le monde par des **triplets** (sujet, propriété, objet). Exemple en Turtle :

```
cin:SamiBouajila a cin:Acteur ; foaf:name "Sami Bouajila" ; cin:joueDans cin:UnFils .
```

Le mot `a` est un raccourci de `rdf:type`, le point-virgule poursuit le même sujet et le point termine le bloc. Les préfixes `cin:` (notre vocabulaire) et `foaf:` (vocabulaire FOAF réutilisé) sont des abréviations d'adresses, pas des classes.

2.1 Classes (6)

Classe	Description	Hiérarchie
cin:Personne	Toute personne du monde du cinéma	sous-classe de foaf:Person
cin:Realisateur	Personne qui réalise des films	sous-classe de cin:Personne
cin:Acteur	Personne qui joue dans des films	sous-classe de cin:Personne
cin:Film	Un film	-
cin:Genre	Genre d'un film (Drame, Thriller...)	-
cin:Salle	Salle de cinéma	-

À ne pas confondre : `cin:Personne` est définie dans notre `schema.ttl`, alors que `foaf:Person` est définie dans FOAF et seulement citée après `rdfs:subClassOf`.

2.2 Propriétés (9) avec domain et range

`rdfs:domain` est le type du sujet (début de la flèche), `rdfs:range` le type de l'objet (fin de la flèche).

Propriété	Domain	Range	Remarque
cin:participeA	Personne	Film	super-propriété
cin:realise	Realisateur	Film	sous-propriété de participeA
cin:joueDans	Acteur	Film	sous-propriété de participeA
cin:aGenre	Film	Genre	

Propriété	Domain	Range	Remarque
cin:projeteDans	Film	Salle	
cin:titre	Film	xsd:string	
cin:anneeSortie	Film	xsd:integer	
cin:duree	Film	xsd:integer	en minutes
cin:ville	Salle	xsd:string	

2.3 Vocabulaire réutilisé

Je réutilise FOAF : `foaf:name` pour le nom des personnes et `foaf:Person` comme super-classe de `cin:Personne`. On évite ainsi d'inventer une propriété `cin:nom`, et les données restent compréhensibles par d'autres applications.

2.4 Schéma graphique du modèle

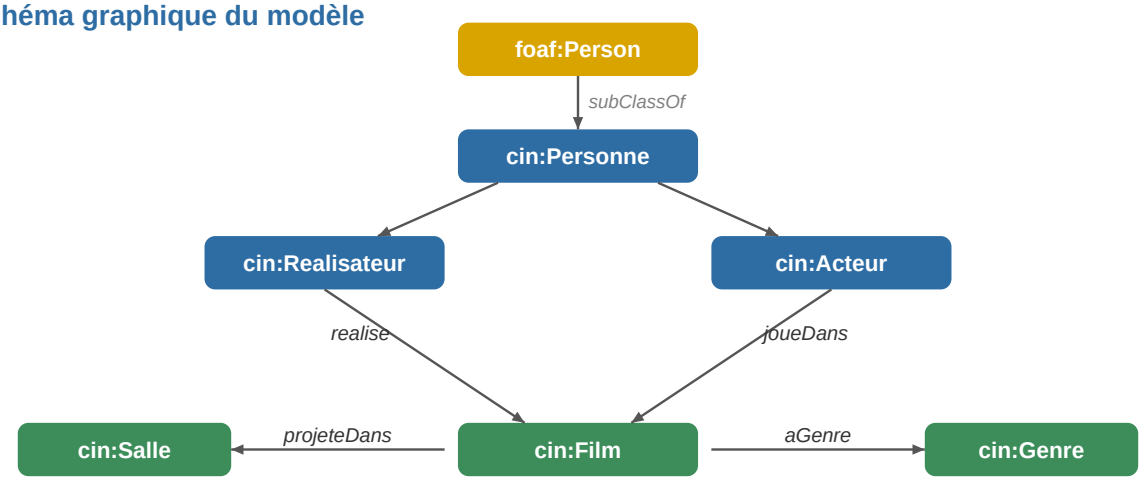


Figure 1 : schéma du vocabulaire (classes en bleu, super-classe FOAF en jaune, classes liées au film en vert). realise et joueDans sont des sous-propriétés de participeA.

3. Étape 2 : requêtes SPARQL

SPARQL pose une question au graphe : on écrit un motif ressemblant à un triplet, avec des variables (précédées de **?**), et le moteur renvoie tout ce qui correspond. Dix requêtes sont écrites dans `queries.py` ; les valeurs saisies par l'utilisateur sont échappées avant d'être insérées dans la requête.

N°	Question métier	Éléments SPARQL
Q1	Quels films (titre, année, durée) ?	SELECT, FILTER, ORDER BY
Q2	Quels films d'un genre donné ?	jointure, FILTER, LCASE
Q3	Combien de films par réalisateur ?	GROUP BY, COUNT
Q4	Quels films joue tel acteur ?	FILTER, CONTAINS
Q5	Combien de films par salle ?	OPTIONAL, COUNT, GROUP BY
Q6	Détail d'un film avec ses genres	GROUP_CONCAT
Q7	Équipe d'un film (réalisateur et acteurs)	BIND, IF, EXISTS (inférence)
Q8	Toutes les personnes (foaf:Person)	inférence nécessaire
Q9	Tous les acteurs (cin:Acteur)	inférence nécessaire
Q10	Toutes les participations (cin:participeA)	inférence nécessaire

Exemple Q3 : nombre de films par réalisateur

```
SELECT ?nom (COUNT(?film) AS ?nbFilms) WHERE {
  ?r a cin:Realisateur ; foaf:name ?nom ; cin:realise ?film .
} GROUP BY ?nom ORDER BY DESC(?nbFilms) ?nom
```

Avant le regroupement, il y a une ligne par couple (réalisateur, film) ; `GROUP BY` rassemble les lignes d'un même nom et `COUNT` les compte. **Résultat obtenu** : Kaouther Ben Hania : 2 films ; Mehdi Barsaoui : 1 film ; Mohamed Ben Attia : 1 film.

Exemple Q5 : films projetés par salle (`OPTIONAL` conserve les salles sans film)

```
SELECT ?id ?nom ?ville (COUNT(?film) AS ?nbFilms) WHERE {
  ?id a cin:Salle ; rdfs:label ?nom ; cin:ville ?ville .
  OPTIONAL { ?film cin:projeteDans ?id . }
} GROUP BY ?id ?nom ?ville ORDER BY ?nom
```

Une requête de contrôle, `SELECT ?titre WHERE { ?f cin:titre ?titre }`, a renvoyé les quatre titres du projet : Un fils, La Belle et la Meute, Hedi, Les Filles d'Olfa. Un film d'essai ajouté temporairement dans `data.ttl` est apparu dès le lancement suivant, ce qui confirme que le graphe relit les données à chaque exécution (il a ensuite été retiré pour garder les chiffres du rapport).

4. Étape 3 : inférence RDFS

Python charge `schema.ttl` et `data.ttl` dans un graphe (`graph.py`). L'inférence est faite par **owlrl** (sémantique RDFS) sur une **copie** du graphe : on obtient deux graphes comparables, `G_BRUT` (ce que j'ai écrit) et `G_INFERE` (enrichi).

4.1 Règles appliquées

Règle RDFS	Principe	Exemple dans les données
rdfs:subClassOf	x est de type C et C est sous-classe de D, donc x est de type D	Sami Bouajila (Acteur) devient aussi Personne et foaf:Person
rdfs:subPropertyOf	x p y et p est sous-propriété de q, donc x q y	x realise f ou x joueDans f, donc x participeA f
rdfs:domain	x p y et domain(p) = C, donc x est de type C	Hend Sabri n'a pas de type ; comme elle joue dans un film, elle devient Acteur

4.2 Exemple : Hend Sabri

Dans `data.ttl`, Hend Sabri n'a aucun type : le graphe brut ne contient que `foaf:name` et `cin:joueDans cin:LesFillesDOlfa`. Le graphe inféré ajoute :

- `cin:participeA cin:LesFillesDOlfa` (règle subPropertyOf) ;
- `rdf:type cin:Acteur` (règle domain : le domaine de joueDans est Acteur) ;
- `rdf:type cin:Personne` puis `rdf:type foaf:Person` (règle subClassOf).

Chaîne de déduction : **joueDans → Acteur → Personne → foaf:Person**. Elle devient actrice sans que ce soit écrit dans les données.

4.3 Comparaison avec et sans inférence

Mesure	Sans inférence	Avec inférence	Explication
Nombre de triplets	121	287	166 triplets déduits
Personnes (foaf:Person)	0	10	3 réalisateurs + 6 acteurs + Hend Sabri
Acteurs (cin:Acteur)	6	7	Hend Sabri déduite par domain
Participations (cin:participeA)	0	11	4 réalisations + 7 interprétations

Commentaire. Sans inférence, les requêtes Q7, Q8 et Q10 ne donnent aucun résultat, car les données n'utilisent que les classes et propriétés les plus précises (Acteur, Realisateur, joueDans, realise). Avec l'inférence, les connaissances implicites deviennent explicites. Le coût est une augmentation du nombre de triplets, mais les requêtes deviennent plus simples (une seule propriété `participeA` au lieu de deux) et plus complètes.

5. Étape 4 : web service REST

Le service est développé avec Flask (`app.py`). Toutes les réponses sont en JSON, accents conservés. Il expose 8 endpoints (5 étaient demandés), tous en GET.

Endpoint	Rôle	Paramètres
/api/films	Liste des films	annee_min, duree_min, inference
/api/films/<id>	Détail d'un film et son équipe	id du film (ex. UnFils)
/api/genres/<nom>/films	Films d'un genre	nom du genre
/api/realisateurs	Réalisateurs et nombre de films	inference
/api/acteurs/<nom>/films	Films d'un acteur	nom (partiel accepté)
/api/salles	Salles et nombre de films	ville
/api/personnes	Personnes FOAF	inference=false ou true

Endpoint	Rôle	Paramètres
/api/inference/comparaison	Comptages avec / sans inférence	-

Exemple de réponse : `GET /api/films/UnFils`

```
{"id": "UnFils", "titre": "Un fils", "annee": 2019, "duree": 96,
 "genres": "Drame",
 "equipe": [
  {"nom": "Mehdi Barsaoui", "role": "Réalisateur"},
  {"nom": "Najla Ben Abdallah", "role": "Acteur"},
  {"nom": "Sami Bouajila", "role": "Acteur"}]}
```

Gestion des erreurs (réponse toujours en JSON, au format `{"erreur": "..."}`) :

- **400** : paramètre invalide (ex. `/api/films?annee_min=abc` : le paramètre doit être un entier positif) ;
- **404** : ressource inconnue (ex. `/api/films/Inconnu`) ;
- **500** : erreur interne.

Structure du code et lancement

Fichier	Rôle
schema.ttl	Vocabulaire RDFS (classes, propriétés, hiérarchies, domain/range)
data.ttl	Données RDF (films, personnes, genres, salles)
graph.py	Chargement du graphe et application de l'inférence
queries.py	Les 10 requêtes SPARQL
app.py	Routes REST, paramètres, gestion des erreurs
demo_inference.py	Affichage de la comparaison avec / sans inférence

```
pip install -r requirements.txt
python demo_inference.py   # comparaison avec / sans inférence
python app.py              # service sur http://127.0.0.1:5000
```

6. Difficultés rencontrées

Problème	Cause	Solution
Lignes « ... » dans la console Python	La console attend la suite d'un bloc (espaces en début de ligne)	Ctrl + C, puis retaper sans espace initial
Commande python tapée après >>>	Elle se tape dans le terminal, pas dans la console Python	Quitter avec exit() puis lancer depuis le terminal
ModuleNotFoundError: rdflib	Environnement virtuel non activé ou pip install oublié	Activer le venv puis pip install -r requirements.txt
Erreur 400 sur annee_min	Valeur non numérique : erreur volontaire	Utiliser un nombre, ex. annee_min=2017

7. Organisation du travail

Ce projet a été réalisé **entièrement seul par Ahmed Toujani** : schéma RDFS, données, inférence, requêtes SPARQL, web service Flask, tests, rapport et présentation.

8. Déclaration d'utilisation de l'IA générative

Une IA générative a été utilisée pour produire le code du projet. Je l'ai relu, testé et compris, et je peux expliquer chaque partie lors de la soutenance.

9. Conclusion

Ce projet couvre toute la chaîne du Web Sémantique : modélisation avec RDFS, interrogation avec SPARQL, enrichissement par inférence, puis publication des résultats par une API REST en JSON. L'inférence montre l'intérêt de RDFS : les données restent simples, et les connaissances implicites (121 triplets écrits, 287 après déduction) sont obtenues automatiquement. Perspectives : ajouter davantage de données, utiliser OWL, ou relier la base à des sources externes comme Wikidata.